README

Exercise 1 Circle Output Sketch

Real-Time Gesture-Based Regression Output Visualization Using Wekinator, OSC Communication, Circle Size, and Outline Transparency

`CircleOutput.pde`

Contents

1 Purpose of This Processing Sketch **3**

2 Role of This Sketch in Exercise 1 **3**

3 Required Processing Libraries **3**
3.1 oscP5 . . . 3
3.2 netP5 . . . 4

4 Full Code **4**

5 General Structure of the Code **5**

6 Line-by-Line Explanation **6**
6.1 Header Comment . . . 6
6.2 Importing Libraries . . . 7
6.3 Declaring the OSC Object . . . 8
6.4 Raw Wekinator Output Variables . . . 8
6.5 Mapped Visual Variables . . . 8
6.6 Font Variable . . . 9

7 The `setup()` Function **9**
7.1 Creating the Window . . . 9
7.2 Enabling Smooth Drawing . . . 9
7.3 Creating and Applying the Font . . . 10
7.4 Starting the OSC Receiver . . . 10

8 The `draw()` Function **10**
8.1 Clearing the Screen . . . 11
8.2 Constraining the Raw Output Values . . . 11
8.3 Mapping Output 1 to Circle Size . . . 11
8.4 Mapping Output 2 to Outline Alpha . . . 12
8.5 Drawing the Circle . . . 12

9 Displaying Debug Information **13**
9.1 Displaying the Raw Wekinator Outputs . . . 14
9.2 Displaying the Mapped Visual Values . . . 15

10 The `oscEvent()` Function **15**
10.1 Checking the OSC Address . . . 15
10.2 Checking That Two Values Were Received . . . 16
10.3 Reading Output 1 . . . 16
10.4 Reading Output 2 . . . 17
10.5 Printing Received Values to the Console . . . 17

11 What the Sketch Receives from Wekinator **17**

12 Output Mapping Explained **18**
12.1 Circle Size Mapping . . . 18
12.2 Alpha Mapping . . . 18

13 Compatibility with Exercise 1 Requirements **19**

14 Expected Behavior When Running the Sketch **19**

15 Correct Wekinator Configuration for This Output Sketch **20**

16 Testing Checklist **20**
16.1 Check That the Sketch Runs . . . 20
16.2 Check the OSC Port . . . 20
16.3 Check Wekinator Output Settings . . . 20
16.4 Check the Number of Outputs . . . 20
16.5 Check the Visual Mapping . . . 20
16.6 Check the Console . . . 21

17 Why This Sketch Is Suitable for Regression **21**

18 Important Design Choices **21**
18.1 Centered Circle . . . 21
18.2 No Fill . . . 21
18.3 White Stroke with Variable Alpha . . . 22
18.4 Safe Value Range . . . 22
18.5 Debug Text . . . 22

19 Relationship Between This Sketch and the Input Sketch **22**

20 Summary **23**

1. Purpose of This Processing Sketch

This Processing sketch is the **output / visualization sketch** for **Lab 3 — Exercise 1: Gesture-Based Regression with Wekinator**.

The purpose of this sketch is to receive the two continuous regression outputs predicted by **Wekinator** and convert them into a visual result. The visual result is a circle whose appearance changes continuously in real time.

The two values received from Wekinator are used as follows:

1. **Output 1** controls the **circle size**.
2. **Output 2** controls the **outline transparency / alpha** of the circle.

This sketch does not collect mouse data and does not train a machine learning model. It only receives the trained model's predictions from Wekinator and visualizes them.

2. Role of This Sketch in Exercise 1

Exercise 1 requires two Processing sketches and one Wekinator project.

The complete system works like this:

```
Mouse gesture in input sketch
        ↓
Input sketch extracts x, y, and speed
        ↓
Input sketch sends 3 normalized values to Wekinator
        ↓
Wekinator uses a regression model
        ↓
Wekinator predicts 2 continuous outputs
        ↓
This output sketch receives those 2 outputs
        ↓
The circle size and outline transparency change in real time
```

This output sketch is responsible for the final visualization step.

It receives OSC messages from Wekinator using:

```
OSC address: /wek/outputs
Listening port: 12000
```

The two received values are expected to be normalized between:

```
0.0 and 1.0
```

Then the sketch maps those normalized values to visual ranges that are useful for drawing.

3. Required Processing Libraries

The sketch uses these two Processing libraries:

```
import oscP5.*;
import netP5.*;
```

3.1. oscP5

`oscP5` is the Processing library used for OSC communication.

OSC means **Open Sound Control**. It is commonly used in multimedia and interactive systems because

it allows programs to send and receive real-time numerical data.

In this sketch, `oscP5` allows Processing to receive the regression outputs sent by Wekinator.

3.2. netP5

`netP5` is often used together with `oscP5` for network communication.

In this specific sketch, the output sketch mainly listens for OSC messages, but importing `netP5` is still useful because it belongs to the standard OSC communication setup in Processing.

4. Full Code

```
/**
 * Lab 3 - Exercise 1
 * Gesture-Based Regression with Wekinator
 *
 * OUTPUT / VISUALIZATION SKETCH
 *
 * This Processing sketch receives 2 continuous regression outputs
 * from Wekinator and uses them to control a circle:
 *
 * Output 1: circle size
 * Output 2: outline transparency / alpha
 *
 * OSC message address received from Wekinator: /wek/outputs
 * Listening port: 12000
 *
 * Recommended Wekinator setup for this exercise:
 * Inputs: 3  -> x, y, speed
 * Outputs: 2 -> circle size, outline alpha
 * Model type: Regression
 * Algorithm: KNN Regressor, k = 3
 *
 * In Wekinator, set both output ranges from 0.0 to 1.0.
 * This sketch maps those normalized values to useful visual ranges.
 */

import oscP5.*;
import netP5.*;

OscP5 oscP5;

// These variables store the raw output values received from Wekinator.
// They are expected to be normalized values between 0.0 and 1.0.
float circleSizeOutput = 0.5;
float outlineAlphaOutput = 0.5;

// These variables store the mapped visual values used for drawing.
float circleSize;
float outlineAlpha;

PFont f;

void setup() {
  size(640, 480, P2D);
  smooth();

  f = createFont("Courier", 16);
  textFont(f);

  // This sketch listens for Wekinator's regression outputs.
  // Wekinator sends output messages to port 12000 by default.
  oscP5 = new OscP5(this, 12000);
```

```
}

void draw() {
  background(0);

  // Keep received values inside the expected normalized range.
  circleSizeOutput = constrain(circleSizeOutput, 0, 1);
  outlineAlphaOutput = constrain(outlineAlphaOutput, 0, 1);

  // Map Wekinator's normalized outputs to visual drawing ranges.
  // Output 1 controls the diameter of the circle.
  circleSize = map(circleSizeOutput, 0, 1, 30, 350);

  // Output 2 controls the alpha/transparency of the circle outline.
  outlineAlpha = map(outlineAlphaOutput, 0, 1, 0, 255);

  // Draw the circle in the center of the window.
  noFill();
  stroke(255, outlineAlpha);
  strokeWeight(6);
  ellipse(width / 2, height / 2, circleSize, circleSize);

  // Display debugging information on the screen.
  fill(255);
  noStroke();
  text("Lab 3 - Exercise 1 Output Sketch", 10, 30);
  text("Receiving 2 regression outputs from Wekinator", 10, 55);
  text("OSC address: /wek/outputs", 10, 80);
  text("Listening port: 12000", 10, 105);

  text("Output 1 - circle size raw     = " + nf(circleSizeOutput, 1, 3), 10, 150);
  text("Output 2 - outline alpha raw   = " + nf(outlineAlphaOutput, 1, 3), 10, 175);
  text("Mapped circle size             = " + nf(circleSize, 1, 1), 10, 215);
  text("Mapped outline alpha           = " + nf(outlineAlpha, 1, 1), 10, 240);
}

// This function is called automatically whenever an OSC message is received.
void oscEvent(OscMessage theOscMessage) {
  if (theOscMessage.checkAddrPattern("/wek/outputs")) {
    if (theOscMessage.typetag().length() >= 2) {
      circleSizeOutput = theOscMessage.get(0).floatValue();
      outlineAlphaOutput = theOscMessage.get(1).floatValue();

      println("Received from Wekinator -> size: " + circleSizeOutput
            + " alpha: " + outlineAlphaOutput);
    }
  }
}
```

5. General Structure of the Code

The code has four main parts:

1. **Comments and documentation**
2. **Library imports and global variables**
3. **The `setup()` function**
4. **The `draw()` function**
5. **The `oscEvent()` function**

The most important part for Exercise 1 is the OSC receiving part:

```
void oscEvent(OscMessage theOscMessage)
```

This function receives Wekinator's two outputs and stores them in variables.

Then the `draw()` function uses those variables to draw the circle.

6. Line-by-Line Explanation

6.1. Header Comment

```
/**
```

This begins a multi-line comment.

A multi-line comment is used to describe the program. Processing ignores it during execution, but it is useful for explaining the code to humans.

```
 * Lab 3 - Exercise 1
```

This identifies the lab exercise that the sketch belongs to.

```
 * Gesture-Based Regression with Wekinator
```

This identifies the topic of the exercise.

The system is gesture-based because the complete project begins with mouse gesture data. It uses regression because Wekinator predicts continuous numerical values instead of fixed class labels.

```
 *
 * OUTPUT / VISUALIZATION SKETCH
```

This states that the current Processing sketch is the output sketch.

It is not the data acquisition sketch. It is the sketch that visualizes the results predicted by Wekinator.

```
 *
 * This Processing sketch receives 2 continuous regression outputs
 * from Wekinator and uses them to control a circle:
```

These lines explain the main purpose of the sketch.

The sketch receives two continuous values from Wekinator. Continuous values mean the numbers can change smoothly, such as:

```
0.12
0.37
0.59
0.84
```

They are not discrete categories such as `class 1`, `class 2`, or `class 3`.

```
 *
 * Output 1: circle size
 * Output 2: outline transparency / alpha
```

These lines define the meaning of the two outputs.

The first Wekinator output controls the diameter of the circle.

The second Wekinator output controls the transparency of the circle outline.

In Processing, transparency is commonly called **alpha**.

```
 *
 * OSC message address received from Wekinator: /wek/outputs
 * Listening port: 12000
```

These lines define how this sketch receives data.

The sketch listens for OSC messages with the address:

```
/wek/outputs
```

It listens on port:

```
12000
```

This is the standard output port commonly used by Wekinator for sending predictions to another program.

```
 *
 * Recommended Wekinator setup for this exercise:
 * Inputs: 3  -> x, y, speed
 * Outputs: 2 -> circle size, outline alpha
 * Model type: Regression
 * Algorithm: KNN Regressor, k = 3
```

These lines summarize the correct Wekinator configuration for this exercise.

The complete system uses:

```
3 inputs  → x, y, speed
2 outputs → circle size, outline alpha
```

The machine learning model should be regression, not classification.

The algorithm should be KNN regression with:

```
k = 3
```

```
 *
 * In Wekinator, set both output ranges from 0.0 to 1.0.
 * This sketch maps those normalized values to useful visual ranges.
 */
```

These lines explain an important design choice.

Wekinator outputs should be set between `0.0` and `1.0`. The sketch then converts these values to ranges that are more useful for drawing.

For example:

```
0.0 to 1.0  →  30 to 350 pixels for circle size
0.0 to 1.0  →  0 to 255 for alpha transparency
```

The final line:

```
*/
```

closes the multi-line comment.

6.2. Importing Libraries

```
import oscP5.*;
```

This imports the `oscP5` library.

This library is necessary because the sketch receives OSC messages from Wekinator.

Without this import, Processing would not understand classes such as:

```
OscP5
```

```
OscMessage
```

```
import netP5.*;
```

This imports the `netP5` library.

`netP5` provides network-related support for OSC communication.

Even though this sketch mainly receives data, `netP5` is part of the common OSC setup used with `oscP5`.

6.3. Declaring the OSC Object

```
OscP5 oscP5;
```

This declares a variable named `oscP5`.

Its type is `OscP5`.

This object manages the OSC communication system in the sketch.

In this output sketch, the `oscP5` object listens for messages coming from Wekinator.

6.4. Raw Wekinator Output Variables

```
// These variables store the raw output values received from Wekinator.
// They are expected to be normalized values between 0.0 and 1.0.
float circleSizeOutput = 0.5;
float outlineAlphaOutput = 0.5;
```

The two comment lines explain that these variables store the original values received from Wekinator.

```
float circleSizeOutput = 0.5;
```

This creates a floating-point variable named `circleSizeOutput`.

It stores the first output received from Wekinator.

This value controls the circle size.

It starts at `0.5`, which means the sketch begins with a middle value before any OSC message is received.

Using `0.5` as the starting value is useful because it avoids starting with an invisible or extremely small circle.

```
float outlineAlphaOutput = 0.5;
```

This creates a floating-point variable named `outlineAlphaOutput`.

It stores the second output received from Wekinator.

This value controls the transparency of the circle outline.

It also starts at `0.5`, giving the outline a medium transparency before real values arrive.

Both variables are expected to stay in this range:

```
0.0 to 1.0
```

6.5. Mapped Visual Variables

```
// These variables store the mapped visual values used for drawing.
float circleSize;
float outlineAlpha;
```

These variables store the converted values used by Processing for drawing.

```
float circleSize;
```

This variable stores the actual diameter of the circle in pixels.

It is not limited to `0.0` to `1.0`. Instead, it will be mapped to a visual range:

```
30 to 350 pixels
```

```
float outlineAlpha;
```

This variable stores the actual alpha value used in the `stroke()` function.

In Processing, alpha usually ranges from:

```
0 to 255
```

where:

```
0   = completely transparent
255 = completely opaque
```

6.6. Font Variable

```
PFont f;
```

This declares a font variable named `f`.

The sketch uses this font to display debugging information on screen.

The displayed text helps the user check whether OSC messages are being received correctly.

7. The `setup()` Function

```
void setup() {
```

`setup()` is a special Processing function.

It runs only once when the sketch starts.

It is used to prepare the Processing window, font, and OSC receiver.

7.1. Creating the Window

```
  size(640, 480, P2D);
```

This creates the sketch window.

The window size is:

```
width  = 640 pixels
height = 480 pixels
```

The third parameter, `P2D`, tells Processing to use the 2D renderer.

This is suitable because the visualization only needs 2D drawing: a circle and text.

7.2. Enabling Smooth Drawing

```
  smooth();
```

This enables anti-aliasing.

Anti-aliasing makes the circle outline smoother and less pixelated.

This improves the visual quality of the sketch.

7.3. Creating and Applying the Font

```
  f = createFont("Courier", 16);
```

This creates a font object.

The font is:

```
Courier
```

The font size is:

```
16
```

Courier is a monospaced font, meaning every character has the same width. This makes numerical debugging text easier to read.

```
  textFont(f);
```

This applies the font to the sketch.

All text drawn after this line will use the `Courier` font at size `16`.

7.4. Starting the OSC Receiver

```
  // This sketch listens for Wekinator's regression outputs.
  // Wekinator sends output messages to port 12000 by default.
  oscP5 = new OscP5(this, 12000);
```

The comments explain the purpose of this line.

```
oscP5 = new OscP5(this, 12000);
```

This creates the OSC receiver.

The number `12000` is the port that this Processing sketch listens to.

That means Wekinator must send its outputs to:

```
Port 12000
```

The keyword `this` refers to the current Processing sketch. It tells `oscP5` where to send incoming OSC messages.

When a correct OSC message arrives, Processing automatically calls the `oscEvent()` function.

```
}
```

This closes the `setup()` function.

8. The `draw()` Function

```
void draw() {
```

`draw()` is a special Processing function.

It runs continuously in a loop after `setup()` finishes.

In this sketch, `draw()` is responsible for:

1. Clearing the screen.
2. Constraining the received Wekinator values.
3. Mapping those values to drawing ranges.
4. Drawing the circle.

5. Displaying debugging information.

8.1. Clearing the Screen

```
background(0);
```

This fills the entire window with black.

The value `0` means black in grayscale Processing color.

This is done every frame so that the circle is redrawn cleanly according to the newest Wekinator values.

8.2. Constraining the Raw Output Values

```
// Keep received values inside the expected normalized range.
circleSizeOutput = constrain(circleSizeOutput, 0, 1);
outlineAlphaOutput = constrain(outlineAlphaOutput, 0, 1);
```

The comment explains that the sketch keeps the received values inside the expected normalized range.

```
circleSizeOutput = constrain(circleSizeOutput, 0, 1);
```

This makes sure the first output never goes below `0` and never goes above `1`.

The `constrain()` function works like this:

```
constrain(value, minimum, maximum)
```

So this line means:

```
Keep circleSizeOutput between 0 and 1.
```

```
outlineAlphaOutput = constrain(outlineAlphaOutput, 0, 1);
```

This does the same thing for the second Wekinator output.

It keeps `outlineAlphaOutput` between `0` and `1`.

This is important because even if Wekinator produces a value slightly outside the expected range, the sketch will still behave safely.

8.3. Mapping Output 1 to Circle Size

```
// Map Wekinator's normalized outputs to visual drawing ranges.
// Output 1 controls the diameter of the circle.
circleSize = map(circleSizeOutput, 0, 1, 30, 350);
```

The comments explain that the normalized Wekinator output is converted to a drawing range.

```
circleSize = map(circleSizeOutput, 0, 1, 30, 350);
```

The `map()` function converts a value from one range to another range.

The structure is:

```
map(value, inputMin, inputMax, outputMin, outputMax)
```

Here the input range is:

```
0 to 1
```

The output range is:

```
30 to 350
```

So the mapping means:

```
circleSizeOutput = 0.0 → circleSize = 30 pixels
circleSizeOutput = 0.5 → circleSize = 190 pixels
circleSizeOutput = 1.0 → circleSize = 350 pixels
```

This makes the first Wekinator output control the circle diameter.

The minimum value is `30` instead of `0` so the circle does not disappear completely when the output is very low.

8.4. Mapping Output 2 to Outline Alpha

```
  // Output 2 controls the alpha/transparency of the circle outline.
  outlineAlpha = map(outlineAlphaOutput, 0, 1, 0, 255);
```

The comment explains that the second output controls transparency.

```
outlineAlpha = map(outlineAlphaOutput, 0, 1, 0, 255);
```

This maps the normalized second output to Processing's alpha range.

The input range is:

```
0 to 1
```

The output range is:

```
0 to 255
```

So the mapping means:

```
outlineAlphaOutput = 0.0 → outlineAlpha = 0
outlineAlphaOutput = 0.5 → outlineAlpha = 127.5
outlineAlphaOutput = 1.0 → outlineAlpha = 255
```

In Processing alpha values:

```
0   = fully transparent
255 = fully visible / opaque
```

So the second Wekinator output directly controls how visible the circle outline is.

8.5. Drawing the Circle

```
  // Draw the circle in the center of the window.
  noFill();
```

The comment explains that the sketch draws a circle in the center of the window.

```
noFill();
```

This disables the fill color.

As a result, the circle has no filled interior. Only its outline is drawn.

This is appropriate because the exercise specifically refers to the **transparency of the outline**.

```
  stroke(255, outlineAlpha);
```

This sets the outline color.

The first value, `255`, means white.

The second value, `outlineAlpha`, controls the transparency.

So the circle outline is white, but its visibility depends on the Wekinator output.

Examples:

```
outlineAlpha = 0   → invisible outline
outlineAlpha = 80  → very transparent outline
outlineAlpha = 180 → clearly visible outline
outlineAlpha = 255 → fully visible outline
```

```
  strokeWeight(6);
```

This sets the thickness of the circle outline to `6` pixels.

A thicker outline makes the transparency effect easier to see.

```
  ellipse(width / 2, height / 2, circleSize, circleSize);
```

This draws the circle.

The `ellipse()` function uses this structure:

```
ellipse(x, y, width, height)
```

Here:

```
width / 2
```

places the circle in the horizontal center of the window.

Since the window width is `640`, this gives:

```
640 / 2 = 320
```

```
height / 2
```

places the circle in the vertical center of the window.

Since the window height is `480`, this gives:

```
480 / 2 = 240
```

The last two parameters are:

```
circleSize, circleSize
```

Using the same value for width and height makes the ellipse a perfect circle.

So the line means:

```
Draw a circle at the center of the screen using the Wekinator-controlled size.
```

9. Displaying Debug Information

```
  // Display debugging information on the screen.
  fill(255);
```

The comment explains that the following lines draw text information on the screen.

```
fill(255);
```

This sets the text color to white.

```
  noStroke();
```

This disables outlines for shapes and text-related drawing after the circle is drawn.

It also prevents unwanted stroke effects from affecting other visual elements.

```
  text("Lab 3 - Exercise 1 Output Sketch", 10, 30);
```

This displays the title of the sketch near the top-left corner of the window.

The position is:

```
x = 10
y = 30
```

```
  text("Receiving 2 regression outputs from Wekinator", 10, 55);
```

This text explains what the sketch is doing.

It confirms that the sketch expects two regression outputs.

```
  text("OSC address: /wek/outputs", 10, 80);
```

This displays the OSC address that the sketch expects to receive.

Wekinator should send predictions using:

```
/wek/outputs
```

```
  text("Listening port: 12000", 10, 105);
```

This displays the port number used by the sketch.

Wekinator should send its output to:

```
12000
```

9.1. Displaying the Raw Wekinator Outputs

```
  text("Output 1 - circle size raw     = " + nf(circleSizeOutput, 1, 3), 10, 150);
```

This displays the first raw value received from Wekinator.

The value is stored in:

```
circleSizeOutput
```

The `nf()` function formats the number so it is easier to read.

```
nf(circleSizeOutput, 1, 3)
```

The `3` means the number is displayed with three digits after the decimal point.

Example:

```
0.642
```

```
  text("Output 2 - outline alpha raw   = " + nf(outlineAlphaOutput, 1, 3), 10, 175);
```

This displays the second raw value received from Wekinator.

The value is stored in:

```
outlineAlphaOutput
```

It is also formatted with three digits after the decimal point.

9.2. Displaying the Mapped Visual Values

```
  text("Mapped circle size            = " + nf(circleSize, 1, 1), 10, 215);
```

This displays the actual circle size after mapping.

The value is stored in:

```
circleSize
```

It is displayed with one digit after the decimal point.

Example:

```
190.0
```

```
  text("Mapped outline alpha          = " + nf(outlineAlpha, 1, 1), 10, 240);
```

This displays the actual alpha value after mapping.

The value is stored in:

```
outlineAlpha
```

It is displayed with one digit after the decimal point.

Example:

```
127.5
```

```
}
```

This closes the `draw()` function.

10. The `oscEvent()` Function

```
// This function is called automatically whenever an OSC message is received.
void oscEvent(OscMessage theOscMessage) {
```

The comment explains the purpose of this function.

`oscEvent()` is automatically called by the `oscP5` library whenever the Processing sketch receives an OSC message.

```
OscMessage theOscMessage
```

is the incoming OSC message.

It contains information such as:

```
OSC address
Value 1
Value 2
```

For this exercise, the expected message is:

```
/wek/outputs value1 value2
```

10.1. Checking the OSC Address

```
  if (theOscMessage.checkAddrPattern("/wek/outputs")) {
```

This checks whether the incoming OSC message has the correct address.

The correct address for Wekinator outputs is:

```
/wek/outputs
```

If the message has a different address, the sketch ignores it.

This is important because OSC systems can sometimes receive messages from different sources or with different purposes.

The sketch should only use messages that belong to Wekinator output predictions.

10.2. Checking That Two Values Were Received

```
    if (theOscMessage.typetag().length() >= 2) {
```

This line checks whether the message contains at least two values.

The function:

```
theOscMessage.typetag()
```

returns information about the types of values inside the OSC message.

The `.length()` part checks how many values are included.

The condition:

```
>= 2
```

means:

```
Only continue if the message contains at least two values.
```

This is important because Exercise 1 requires exactly two outputs:

```
Output 1 → circle size
Output 2 → outline alpha
```

If fewer than two values are received, the sketch does not try to read missing data.

This prevents errors.

10.3. Reading Output 1

```
      circleSizeOutput = theOscMessage.get(0).floatValue();
```

This reads the first value from the OSC message.

In programming, index numbering usually starts at `0`.

So:

```
theOscMessage.get(0)
```

means:

```
Get the first value from the OSC message.
```

The function:

```
.floatValue()
```

converts that value into a floating-point number.

The result is stored in:

```
circleSizeOutput
```

This is the value that controls the circle size.

10.4. Reading Output 2

```
      outlineAlphaOutput = theOscMessage.get(1).floatValue();
```

This reads the second value from the OSC message.

```
theOscMessage.get(1)
```

means:

```
Get the second value from the OSC message.
```

The result is converted to a float and stored in:

```
outlineAlphaOutput
```

This is the value that controls the outline transparency.

10.5. Printing Received Values to the Console

```
      println("Received from Wekinator -> size: " + circleSizeOutput
            + " alpha: " + outlineAlphaOutput);
```

This prints the received values to the Processing console.

This is useful for debugging because it allows the user to confirm that messages are actually arriving from Wekinator.

The printed output will look like this:

```
Received from Wekinator -> size: 0.73 alpha: 0.41
```

This means Wekinator sent:

```
circle size output = 0.73
outline alpha output = 0.41
```

The line is split across two lines in the code for readability, but it works as one complete `println()` statement.

```
    }
  }
}
```

These closing braces close:

1. The inner `if` statement that checks the number of values.
2. The outer `if` statement that checks the OSC address.
3. The `oscEvent()` function.

11. What the Sketch Receives from Wekinator

Every time Wekinator sends a prediction, this sketch expects an OSC message like this:

```
Address: /wek/outputs
Arguments:
  1. circleSizeOutput
  2. outlineAlphaOutput
```

Example:

```
/wek/outputs 0.75 0.30
```

This means:

```
Output 1 = 0.75 → draw a relatively large circle
Output 2 = 0.30 → draw a fairly transparent outline
```

The sketch then maps these values:

```
0.75 → circle size between 30 and 350 pixels
0.30 → outline alpha between 0 and 255
```

12. Output Mapping Explained

The two most important mapping lines are:

```
circleSize = map(circleSizeOutput, 0, 1, 30, 350);
outlineAlpha = map(outlineAlphaOutput, 0, 1, 0, 255);
```

12.1. Circle Size Mapping

```
Wekinator output range: 0.0 to 1.0
Processing drawing range: 30 to 350 pixels
```

Examples:

Wekinator Output 1	Mapped Circle Size
0.0	30 px
0.25	110 px
0.5	190 px
0.75	270 px
1.0	350 px

This makes the visual change clear and easy to see.

12.2. Alpha Mapping

```
Wekinator output range: 0.0 to 1.0
Processing alpha range: 0 to 255
```

Examples:

Wekinator Output 2	Mapped Alpha	Visual Result
0.0	0	Invisible outline
0.25	63.75	Very transparent outline
0.5	127.5	Half-transparent outline
0.75	191.25	Mostly visible outline
1.0	255	Fully visible outline

This directly satisfies the visual requirement of controlling the transparency of the circle outline.

13. Compatibility with Exercise 1 Requirements

This sketch satisfies the Exercise 1 output requirements as follows:

Exercise 1 Requirement	How This Sketch Satisfies It
Use a second Processing sketch for visualization	This is the output / visualization sketch
Receive Wekinator regression outputs	Uses `oscEvent(OscMessage theOscMessage)`
Use OSC communication	Uses the `oscP5` library
Receive messages from `/wek/outputs`	Checks `checkAddrPattern("/wek/outputs")`
Listen on port `12000`	Uses `new OscP5(this, 12000)`
Use 2 Wekinator outputs	Reads `get(0)` and `get(1)`
Output 1 controls circle size	Stores first value in `circleSizeOutput` and maps it to `circleSize`
Output 2 controls outline transparency	Stores second value in `outlineAlphaOutput` and maps it to `outlineAlpha`
Show continuous visual changes	Uses continuous float values and redraws every frame
Support regression	Uses smooth numerical outputs instead of class labels

14. Expected Behavior When Running the Sketch

When the sketch starts, a black window appears.

A white circle outline is drawn in the center of the window.

The sketch displays debugging text such as:

```
Lab 3 - Exercise 1 Output Sketch
Receiving 2 regression outputs from Wekinator
OSC address: /wek/outputs
Listening port: 12000

Output 1 - circle size raw     = ...
Output 2 - outline alpha raw   = ...
Mapped circle size             = ...
Mapped outline alpha           = ...
```

Before Wekinator sends data, both raw outputs start at:

```
0.5
```

So the circle begins with a medium size and medium transparency.

After Wekinator starts running and sends predictions, the values update in real time.

Expected behavior:

```
When Wekinator output 1 increases → the circle becomes larger.
When Wekinator output 1 decreases → the circle becomes smaller.
When Wekinator output 2 increases → the outline becomes more visible.
When Wekinator output 2 decreases → the outline becomes more transparent.
```

15. Correct Wekinator Configuration for This Output Sketch

For this sketch to work correctly, Wekinator should be configured as follows:

```
Project type: Regression
Inputs: 3
Outputs: 2
Algorithm: KNN Regressor
k: 3
```

The output settings should be:

```
Output message: /wek/outputs
Output destination port: 12000
Output 1 range: 0.0 to 1.0
Output 2 range: 0.0 to 1.0
```

The meaning of the Wekinator outputs should be:

```
Output 1 → circle size
Output 2 → outline alpha / transparency
```

16. Testing Checklist

16.1. Check That the Sketch Runs

Run the output sketch in Processing.

Expected result:

```
A black window appears.
A circle outline appears in the center.
Debugging text appears in the top-left corner.
```

16.2. Check the OSC Port

The Processing console should show that `oscP5` is listening on port `12000`.

This confirms that the sketch is ready to receive Wekinator outputs.

16.3. Check Wekinator Output Settings

In Wekinator, make sure the output destination is:

```
Host: 127.0.0.1
Port: 12000
Message: /wek/outputs
```

16.4. Check the Number of Outputs

Wekinator must be configured with:

```
2 outputs
```

If Wekinator sends fewer than two values, the sketch will not update correctly.

16.5. Check the Visual Mapping

Move the mouse in the input sketch while Wekinator is running.

Expected result:

```
The circle size changes continuously.
The outline transparency changes continuously.
```

16.6. Check the Console

The Processing console should print messages like:

```
Received from Wekinator -> size: 0.64 alpha: 0.82
```

If these messages appear, the sketch is receiving data successfully.

17. Why This Sketch Is Suitable for Regression

This sketch is suitable for regression because it treats Wekinator outputs as continuous values.

It does not wait for a class label such as:

```
small circle
medium circle
large circle
```

Instead, it receives smooth values such as:

```
0.21
0.22
0.24
0.29
0.35
```

This allows the visual result to change gradually.

That is the key difference between classification and regression:

```
Classification → selects a discrete category
Regression     → predicts continuous numerical values
```

In this exercise, regression is appropriate because the circle size and outline transparency should not jump between fixed categories. They should change smoothly according to the gesture.

18. Important Design Choices

18.1. Centered Circle

The circle is drawn at:

```
width / 2, height / 2
```

This keeps the visualization simple and focused on the two required parameters:

```
size
alpha
```

The circle position does not change because the exercise asks for size and outline transparency, not position.

18.2. No Fill

The line:

```
noFill();
```

makes the circle hollow.

This makes the outline transparency easier to observe.

18.3. White Stroke with Variable Alpha

The line:

```
stroke(255, outlineAlpha);
```

uses white for the circle outline and uses Wekinator's second output to control transparency.

This creates a clear visual relationship between the model output and the displayed result.

18.4. Safe Value Range

The lines:

```
circleSizeOutput = constrain(circleSizeOutput, 0, 1);
outlineAlphaOutput = constrain(outlineAlphaOutput, 0, 1);
```

protect the visualization from unexpected output values.

This makes the sketch more stable during testing.

18.5. Debug Text

The debugging text helps confirm that the sketch is receiving and mapping Wekinator outputs correctly.

This is especially useful during training and testing because the user can see both:

```
Raw Wekinator values
Mapped Processing drawing values
```

19. Relationship Between This Sketch and the Input Sketch

The input sketch and output sketch do different jobs.

The input sketch sends:

```
x position
y position
movement speed
```

to Wekinator.

This output sketch receives:

```
circle size
outline alpha
```

from Wekinator.

So the full mapping learned by Wekinator is:

```
x position + y position + speed
        ↓
circle size + outline transparency
```

This is the core idea of Exercise 1.

The user trains Wekinator by recording examples of mouse gestures and assigning desired output values. After training, Wekinator predicts the circle size and outline transparency for new mouse gestures.

20. Summary

This Processing sketch is the output visualization component of Exercise 1.

It performs these main tasks:

1. Opens a 640 × 480 Processing window.
2. Listens for OSC messages from Wekinator on port `12000`.
3. Accepts messages with the OSC address `/wek/outputs`.
4. Reads two continuous regression outputs.
5. Uses the first output to control circle size.
6. Uses the second output to control outline transparency.
7. Draws a centered circle that updates continuously.
8. Displays debugging information to verify the received and mapped values.

This sketch is fully compatible with Exercise 1 because it receives exactly the two required Wekinator outputs and maps them to the two required visual parameters:

```
Output 1 → circle size
Output 2 → outline transparency / alpha
```

Together with the input sketch and the Wekinator KNN regression model, this sketch completes the gesture-based regression system required for the exercise.